



Universidad Nacional Autónoma de México

Facultad de Ingeniería

Materia: Organización Y Arquitectura de Computadoras

Profesor: M.I. José Antonio de Jesús Arredondo Garza

Práctica 2

Sistema de control de segundo orden

No. de Cuenta	**Nombre**
320068234	Ayala Hernández María Fernanda
320335163	Salazar Islas Luis Daniel
320078282	Tepal Briseño Hansel Yael
320223822	Ugartechea González Luis Antonio

Fecha de entrega: 12 de Octubre del 2026

Calificación: _____

1 Objetivo

En esta práctica se implementó en una Raspberry Pi Pico 2 W un sistema de control de segundo orden en tiempo discreto. El sistema lee la posición de un potenciómetro mediante el convertidor analógico-digital (ADC), calcula la salida usando una ecuación de diferencias y ejecuta el proceso cada $T_s = 0.025$ s, equivalente a una frecuencia de muestreo de $f_s = 40$ Hz.

El objetivo es relacionar la función de transferencia continua con una implementación digital en MicroPython y observar el comportamiento del sistema directamente sobre el hardware.

2 Fundamentos teóricos

Una función de transferencia de segundo orden puede escribirse como

$$H(s) = \frac{\omega_0}{s^2 + 2\omega_0 s + \omega_0^2}. \tag{1}$$

Para trabajar con la Raspberry Pi Pico 2 W, el sistema continuo debe discretizarse para obtener una función $H(z)$ y una ecuación de diferencias.

El periodo de muestreo utilizado es

$$T_s = 0.025 \text{ s}, \qquad f_s = \frac{1}{T_s} = 40 \text{ Hz}. \tag{2}$$

La implementación utiliza la forma discreta

$$H(z) = \frac{b_0 + b_1 z^{-1} + b_2 z^{-2}}{1 + a_1 z^{-1} + a_2 z^{-2}}. \tag{3}$$

Despejando la salida se obtiene la ecuación de diferencias

$$y[k] = b_0 x[k] + b_1 x[k-1] + b_2 x[k-2] - a_1 y[k-1] - a_2 y[k-2]. \tag{4}$$

En el programa se emplean los coeficientes

$$b_0 = 0.01, \quad b_1 = 0.02, \quad b_2 = 0.01, \qquad a_1 = -1.6, \quad a_2 = 0.64. \tag{5}$$

Por tanto, el denominador queda como

$$z^2 - 1.6z + 0.64 = (z - 0.8)^2. \tag{6}$$

Los polos tienen magnitud 0.8, por lo que se encuentran dentro del círculo unitario y el filtro discreto es estable. Además, la suma de los coeficientes del numerador es igual a la evaluación del denominador en $z = 1$, por lo que la ganancia de corriente directa es aproximadamente uno.

3 Desarrollo

El programa se divide en cuatro tareas principales:

- configurar el ADC del pin 26 para leer el potenciómetro;
- suavizar la lectura mediante un filtro exponencial;
- calcular la salida del sistema con la ecuación de diferencias;
- ejecutar las tareas periódicamente y mostrar la muestra actual.

El ADC de la Pico entrega valores entre 0 y 65535. Para reducir las variaciones rápidas del potenciómetro se utiliza un filtro de promedio exponencial:

$$x_f[k] = \alpha x_{ADC}[k] + (1-\alpha)x_f[k-1], \qquad \alpha = 0.1. \tag{7}$$

Un valor pequeño de α produce una lectura más suave, aunque también incrementa el tiempo de respuesta ante cambios del potenciómetro.

3.1 Estados del sistema

Las variables `x_k1` y `x_k2` almacenan las dos entradas anteriores, mientras que `y_k1` y `y_k2` almacenan las dos salidas anteriores. Al terminar cada iteración, los estados se desplazan para que estén disponibles en el siguiente periodo de muestreo.

3.2 Temporizador e interrupción

El temporizador se configura con un periodo de 25 ms:

```
1 timer_muestreo.init(period=25,
2                     mode=Timer.PERIODIC,
3                     callback=iteracion_control)
```

Cada interrupción ejecuta `iteracion_control`, lee el ADC, actualiza el filtro y calcula la salida. La interrupción no imprime directamente en la terminal: solo guarda la última muestra. El bucle principal se encarga de imprimirla, evitando operaciones de salida dentro de la interrupción.

3.3 Protecciones de estabilidad

El programa verifica que la salida sea finita mediante `isfinite`. Si se detecta `nan` o infinito, se reinician los estados de salida. También se limita la salida al intervalo del ADC, $[-65535, 65535]$, para evitar que una condición anormal se propague al resto del sistema.

4 Implementación en MicroPython

El código completo utilizado en la Raspberry Pi Pico 2 W es el siguiente.

```
'''
SISTEMA DE CONTROL DE SEGUNDO ORDEN

Esta implementación ejecuta una función de transferencia discreta H(z)
basada en un sistema de segundo orden continuo.
Se utiliza un temporizador por hardware de la Raspberry Pi Pico 2W
para garantizar un tiempo de muestreo estricto de Ts = 0.025s (40 Hz).

Por:
- Ayala Hernández María Fernanda
- Salazar Islas Luis Daniel
- Tepal Briseño Hansel Yael
- Ugartechea González Luis Antonio
'''

# --- EXPLICACIÓN DEL FUNCIONAMIENTO ---
# El script utiliza el módulo `machine.Timer` de MicroPython para
# generar una interrupción de hardware exacta cada 25 milisegundos (40 Hz),
# cumpliendo con la restricción de muestreo sin bloquear el microcontrolador.
# En cada interrupción (ciclo de reloj) el sistema realiza lo siguiente:
# 1. Lee la entrada actual x[k] (ej. desde un pin analógico).
# 2. Calcula la salida actual y[k] resolviendo la ecuación de diferencias obtenida al discretizar H(s) a H(z).
# 3. Actualiza el historial de variables (memoria) desplazando los valores anteriores (x[k-1], y[k-1], etc.) para que estén listos en el próximo ciclo.
# --------------- ARM Assembly function ---------------

from machine import Pin, Timer, ADC
from math import isfinite
import time

# --- COEFICIENTES DEL SISTEMA DISCRETO H(z) ---
b0 = 0.01
b1 = 0.02
b2 = 0.01
a1 = -1.6
a2 = 0.64

# --- VARIABLES DE ESTADO (Memoria del sistema) ---
```

```
x_k1 = 0.0
x_k2 = 0.0
y_k1 = 0.0
y_k2 = 0.0
entrada_filtrada = 0.0
ultima_entrada = 0
ultima_salida = 0.0
nueva_muestra = False

ALFA_ENTRADA = 0.1
LIMITE_SALIDA = 65535.0

# --- CONFIGURACIÓN DE PINES ---
entrada_adc = ADC(26)

def iteracion_control(timer):
    global x_k1, x_k2, y_k1, y_k2, entrada_filtrada
    global ultima_entrada, ultima_salida, nueva_muestra
    # 1. Leer la entrada actual x[k]
    # El ADC tiene un valor de 0 a 65535
    lectura_adc = entrada_adc.read_u16()
    entrada_filtrada = (ALFA_ENTRADA * lectura_adc) + ((1 -
       ALFA_ENTRADA) * entrada_filtrada)
    x_k = entrada_filtrada

    # 2. Calcular la salida y[k] mediante la ecuación de diferencias
    y_k = (b0 * x_k) + (b1 * x_k1) + (b2 * x_k2) - (a1 * y_k1) - (a2
        * y_k2)

    if not isfinite(y_k):
        y_k = 0.0
        y_k1 = 0.0
        y_k2 = 0.0
    elif y_k > LIMITE_SALIDA:
        y_k = LIMITE_SALIDA
    elif y_k < -LIMITE_SALIDA:
        y_k = -LIMITE_SALIDA

    # Guardar la muestra para imprimirla fuera de la interrupción.
    ultima_entrada = x_k
    ultima_salida = y_k
    nueva_muestra = True

    # 4. Actualizar las variables de estado para la siguiente
       interrupción
    x_k2 = x_k1
    x_k1 = x_k
    y_k2 = y_k1
```

```
    y_k1 = y_k

# --- INICIALIZACIÓN DEL TIMER ---
timer_muestreo = Timer(-1)
timer_muestreo.init(period=25, mode=Timer.PERIODIC, callback=
    iteracion_control)

print("Sistema de control de segundo orden iniciado a 40 Hz...")

# El bucle principal queda libre para otras tareas (pantallas,
    comunicaciones, etc.)
try:
    while True:
        if nueva_muestra:
            nueva_muestra = False
            print(f"\r\x1b[2Kx[k]: {ultima_entrada} | y[k]: {
                ultima_salida}", end="")
        time.sleep_ms(1)
except KeyboardInterrupt:
    # Detener el timer si se cancela la ejecución en la consola
    timer_muestreo.deinit()
    print()
    print("Sistema detenido.")
```

Listing 1: Implementación del sistema de control

5 Conexión del potenciómetro

El potenciómetro se conecta entre 3.3 V y GND, y su terminal central se lleva al pin ADC26. Para disminuir el ruido eléctrico se recomienda colocar un capacitor de 100 nF entre ADC26 y GND, lo más cerca posible de la placa. Una resistencia serie de aproximadamente 1 kΩ puede colocarse entre el cursor del potenciómetro y el ADC para formar un filtro RC junto con el capacitor.

La Pico y el potenciómetro deben compartir la misma referencia de tierra. No debe aplicarse 5 V al ADC, ya que sus entradas trabajan con el nivel de 3.3 V.

6 Ejecución en la Raspberry Pi Pico

El programa se carga y ejecuta desde la carpeta `practica_02` con:

```
../upload.sh source/practica_02.py
```

El script de carga verifica que el archivo exista, libera el puerto serial si está ocupado, copia el programa a la Pico como `main.py` y lo ejecuta mediante `mpremote`.

```bash
#!/usr/bin/env bash

set -euo pipefail

script="${1:?Usage: ./upload.sh SCRIPT.py [DEVICE]}"
device="${2:-/dev/ttyACM0}"

if [[ ! -f "$script" ]]; then
  echo "File not found: $script" >&2
  exit 1
fi

if fuser "$device" >/dev/null 2>&1; then
  echo "Port $device is busy. Stopping the process using it..." >&2
  fuser -k "$device" >/dev/null 2>&1 || true
fi

mpremote connect "$device" fs cp "$script" :main.py
mpremote connect "$device" run "$script"
```

Listing 2: Script utilizado para cargar el programa

7 Resultados

Al iniciar el programa se muestra el mensaje de arranque y después se actualiza una sola línea de la terminal con el valor de la entrada y la salida:

```
Sistema de control de segundo orden iniciado a 40 Hz...
x[k]: valor_de_la_entrada | y[k]: valor_de_la_salida
```

El retorno de carro `\r` evita crear una línea nueva por cada muestra. La lectura se actualiza nominalmente cada 25 ms. El filtro EMA reduce las variaciones rápidas del potenciómetro y los polos de magnitud 0.8 evitan el crecimiento de la respuesta que anteriormente producía valores `nan`.

La salida también puede almacenarse en un archivo desde la terminal:

```
../upload.sh source/practica_02.py >> source/output.txt
```

8 Conclusiones individuales

8.1 Ayala Hernández María Fernanda

La práctica permitió implementar un sistema de segundo orden en tiempo discreto sobre una Raspberry Pi Pico 2 W. El uso del ADC y del temporizador mostró cómo un microcontrolador puede adquirir señales y procesarlas con un periodo de muestreo definido.

8.2 Salazar Islas Luis Daniel

La implementación permitió relacionar una función de transferencia con una ecuación de diferencias ejecutable en MicroPython. También se observó que la elección de los polos es fundamental para evitar que la respuesta digital sea inestable.

8.3 Tepal Briseño Hansel Yael

El filtrado exponencial ayudó a reducir el ruido producido por el potenciómetro. Esta práctica mostró que la estabilidad depende tanto del algoritmo de control como de la calidad de la señal de entrada y de la conexión física del circuito.

8.4 Ugartechea González Luis Antonio

La ejecución en la Raspberry Pi Pico 2 W permitió comprobar la interacción entre el software y el hardware. El temporizador periódico garantiza el ritmo de procesamiento, mientras que la salida en la terminal facilita observar el comportamiento del sistema durante las pruebas.